



25. Python: Multi-Dimensional Lists

"One list is useful. A list of lists can model an entire world."

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 24. Python: Iterators and Generators](#)

Contents:

[1. What is a Multi-Dimensional List?](#)

[Types of Multi-Dimensional Lists](#)

[2. Some Operations on Multi-Dimensional Lists](#)

[2.1. Common Operations](#)

[2.2. Modifying Multi-Dimensional Lists](#)

[2.3. Closest Pair of Points](#)

[2.4. Dot Product](#)

[2.5. Grading System Using a 3-Dimensional List](#)

[Summary](#)

[References](#)

So far, we have worked with ordinary lists:

```
numbers = [10, 20, 30, 40, 50]
```

A normal list stores data in a single sequence. This is useful for many problems, but real-world data is often more complex than a simple row of values.

Think about a spreadsheet. Think about a chessboard. Think about a calendar. Think about the distances between cities. These structures contain rows and columns rather than a single line of information.

To represent such data, Python allows us to create **Multi-Dimensional Lists**—lists that contain other lists.

Instead of storing a single value in each position, a multi-dimensional list can store an entire list, allowing us to organize data into rows, columns, and even higher dimensions.

As the amount of data grows, multi-dimensional lists become one of the most useful tools for representing structured information. They serve as the foundation for matrices, tables, images, game boards, graphs, and many concepts you will encounter later in data science, machine learning, computer graphics, and scientific computing.

1. What is a Multi-Dimensional List?

A **Multi-Dimensional List** is simply a list that contains other lists. Informally, you can think of it as a **list inside another list**.

For example, suppose we want to store the distances between several cities:

```
distances = [
    [0, 983, 787, 714, 1375, 967, 1087],
    [983, 0, 214, 1102, 1505, 1723, 1842],
    [787, 214, 0, 888, 1549, 1548, 1627],
    [714, 1102, 888, 0, 661, 781, 810],
    [1375, 1505, 1549, 661, 0, 1426, 1187],
    [967, 1723, 1548, 781, 1426, 0, 239],
    [1087, 1842, 1627, 810, 1187, 239, 0]
]
```

Each row represents one city, while each column represents the distance to another city.

Another common example is a matrix:

```
matrix = [
    [1, 2, 3, 4, 5],
]
```

```
[6, 7, 0, 0, 0],  
[0, 1, 0, 0, 0],  
[1, 0, 0, 0, 8],  
[0, 0, 9, 0, 3]  
]
```

Notice how every element of the outer list is itself another list. This creates a table-like structure with rows and columns.

Types of Multi-Dimensional Lists

When studying mathematics, computer science, data science, or machine learning, you will frequently encounter several common forms of multi-dimensional data.

- **Vector (1-Dimensional):** A vector is simply a single list of values.

```
vector = [5, 8, 2, 9]
```

You can think of it as one row or one column of data.

- **Matrix (2-Dimensional):** A matrix is a list of lists.

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

Matrices are the most common form of multi-dimensional lists.

- **Tensor (3-Dimensional and Beyond):** A tensor extends the concept even further.

```
tensor = [  
    [  
        [1, 2],
```

```
[3, 4]
],
[
    [5, 6],
    [7, 8]
]
]
```

Tensors are widely used in computer graphics, scientific simulations, and modern Artificial Intelligence systems.

Real-World Examples

Multi-dimensional lists appear everywhere:

- Tables and spreadsheets
- Chessboards and board games
- Calendars
- Seating arrangements
- Images (pixels arranged in rows and columns)
- Maps and geographic data
- Distance tables between cities
- Machine Learning datasets

Whenever information can be organized into rows and columns, a multi-dimensional list is often an appropriate solution.

2. Some Operations on Multi-Dimensional Lists

Now that we know how to create multi-dimensional lists, let's explore some common operations.

2.1. Common Operations

We will use the following city names for the distance table:

```
cities = ["Chicago",
          "Boston",
          "New York",
          "Atlanta",
          "Miami",
          "Dallas",
          "Houston"]
```

- **Displaying the Distance Table:** A distance matrix becomes much easier to understand when displayed as a table.

```
print(f"{'':12}", end="")

for city in cities:
    print(f"{city:12}", end="")

print()

for i in range(len(distances)):
    print(f"{cities[i]:12}", end="")

    for value in distances[i]:
        print(f"{value:12}", end="")

    print()
```

This structure is commonly used in transportation systems, logistics, and graph algorithms.

The following code using the **matrix** we defined earlier:

- **Traversing Every Element:** To visit every value inside a matrix, we use nested loops.

```

for row in matrix:
    for value in row:
        print(value, end=" ")
    print()

```

```

1 2 3 4 5
6 7 0 0 0
0 1 0 0 0
1 0 0 0 8
0 0 9 0 3

```

The outer loop visits each row, while the inner loop visits each value inside that row.

- **Finding the Shape:** The shape of the matrix, which includes the number of rows and number of columns can be easily done by the `len()` method.

```

# Return shape of the matrix
def shape(matrix):
    return len(matrix), len(matrix[0])

print("Length of row:", shape(matrix)[0])
print("Length of column:", shape(matrix)[1])

# Total elements
print("Total elements:", shape(matrix)[0] * shape(matrix)
[1])

```

```

Length of row: 5
Length of column: 5
Total elements: 25

```

- **Finding the Sum of Each Row**

```
for i, row in enumerate(matrix):  
    print(f"Row {i}: {sum(row)}")
```

Row 0: 15

Row 1: 13

Row 2: 1

Row 3: 9

Row 4: 12

- **Finding the Sum of Each Column**

```
# Sum of Columns  
for j in range(len(matrix[0])):  
    column_sum = sum(matrix[i][j] for i in range(len(matrix)))  
    print(f"Column {j}: {column_sum}")
```

Column 0: 8

Column 1: 10

Column 2: 12

Column 3: 4

Column 4: 16

- **Finding the Row and Column with the Largest Sum**

```

largest_row = 0
largest_sum_row = sum(matrix[0])

largest_col = 0
largest_sum_col = sum(matrix[i][0] for i in range(len(matrix)))

for i, row in enumerate(matrix):
    row_sum = sum(row)
    if row_sum > largest_sum_row:
        largest_sum_row = row_sum
        largest_row = i

for j in range(len(matrix[0])):
    column_sum = sum(matrix[i][j] for i in range(len(matrix)))
    if column_sum > largest_sum_col:
        largest_sum_col = column_sum
        largest_col = j

print(f"Largest row: {largest_row}; Sum: {largest_sum_row}")
print(f"Largest column: {largest_col}; Sum: {largest_sum_col}")

```

Largest row: 0; Sum: 15

Largest column: 4; Sum: 16

This technique is commonly used in data analysis and optimization problems.

- **Retrieving the Diagonal Elements:** The main diagonal consists of positions where the row index equals the column index.

```

for i in range(len(matrix)):
    print(matrix[i][i], end=" ")

```

1 7 0 0 3

Diagonal values are important in matrix algebra and graph theory.

- **Sorting Rows:** We can sort the rows according to their row sums.

```
matrix.sort(key=sum)

for row in matrix:
    print(row)
```

```
[0, 1, 0, 0, 0]
[1, 0, 0, 0, 8]
[0, 0, 9, 0, 3]
[6, 7, 0, 0, 0]
[1, 2, 3, 4, 5]
```

Rows with smaller sums will appear first.

- **Sorting Columns:** We can sort the columns according to their column sums.

```
# Sorting Columns
for j in range(len(matrix[0])):
    column = [matrix[i][j] for i in range(len(matrix))]
    column.sort()
    print(column)
```

```
[0, 0, 1, 1, 6]
[0, 0, 1, 2, 7]
[0, 0, 0, 3, 9]
[0, 0, 0, 0, 4]
[0, 0, 3, 5, 8]
```

Likewise, columns with smaller sums will appear first.

- **Transposing:** Transposing a matrix means **turning its rows into columns** (and conversely, its columns into rows). Geometrically, it involves **flipping the matrix across its main diagonal**.

```
# Transpose
def transpose():
    return [[matrix[j][i] for j in range(len(matrix))] for i
in range(len(matrix[0]))]

for row in tranpose():
    print(row)
```

```
[0, 1, 0, 6, 1]
[1, 0, 0, 7, 2]
[0, 0, 9, 0, 3]
[0, 0, 0, 0, 4]
[0, 8, 3, 0, 5]
```

2.2. Modifying Multi-Dimensional Lists

Multi-dimensional lists are still Python lists. This means we can use many normal list operations such as **append**, **replace**, and **reverse**.

However, there is an important detail:

▮ A matrix is only a matrix if every row has the same number of columns.

Once rows have different lengths, the structure becomes a **jagged list** (or ragged list) rather than a true matrix.

- **Appending a New Row:** Adding a new row is straightforward.

```
# Appending new row
matrix.append([8, 8, 8, 8, 8])
```

This is creates a valid matrix because every row contains 5 elements.

```
[0, 1, 0, 0, 0]
[1, 0, 0, 0, 8]
[0, 0, 9, 0, 3]
[6, 7, 0, 0, 0]
[1, 2, 3, 4, 5]
[8, 8, 8, 8, 8]
```

- **Appending a New Column:** To add a column, we must append one value to every row.

```
for i in matrix:  
    i.append(10)
```

The matrix is now 5×6 :

```
[0, 1, 0, 0, 0, 10]  
[1, 0, 0, 0, 0, 8, 10]  
[0, 0, 9, 0, 3, 10]  
[6, 7, 0, 0, 0, 10]  
[1, 2, 3, 4, 5, 10]  
[8, 8, 8, 8, 8, 10]
```

- **Appending a Single Element:** Suppose we do this:

```
matrix[0].append(99)
```

The matrix becomes:

```
[0, 1, 0, 0, 0, 10, 99]  
[1, 0, 0, 0, 0, 8, 10]  
[0, 0, 9, 0, 3, 10]  
[6, 7, 0, 0, 0, 10]  
[1, 2, 3, 4, 5, 10]  
[8, 8, 8, 8, 8, 10]
```

Notice what happened. The first row now contains 6 elements while all other rows contain 5. This is no longer a proper matrix.

It is still a valid multi-dimensional list, but mathematically it becomes a **jagged list** because the rows have different lengths.

- **Replacing Elements:** Replacing values is done using indexing. For example, replace the value 7 with 100:

```
matrix[1][1] = 100
```

The matrix becomes:

```
[0, 1, 0, 0, 0, 10, 99]
[1, 100, 0, 0, 0, 8, 10]
[0, 0, 9, 0, 3, 10]
[6, 7, 0, 0, 0, 10]
[1, 2, 3, 4, 5, 10]
[8, 8, 8, 8, 8, 10]
```

- **Replacing an Entire Row:** A complete row can also be replaced.

```
matrix[2] = [9, 9, 9, 9, 9]
```

The matrix becomes:

```
[0, 1, 0, 0, 0, 10, 99]
[1, 100, 0, 0, 0, 8, 10]
[9, 9, 9, 9, 9]
[6, 7, 0, 0, 0, 10]
[1, 2, 3, 4, 5, 10]
[8, 8, 8, 8, 8, 10]
```

As long as the replacement row has the same length, the matrix structure is preserved.

- **Replacing an Entire Column:** Columns require traversing every row. For example, replace the third column with 99.

```
for i in matrix:
    i[2] = 99
```

The matrix becomes:

```
[0, 1, 99, 0, 0, 10, 99]
[1, 100, 99, 0, 8, 10]
[9, 9, 99, 9, 9]
[6, 7, 99, 0, 0, 10]
[1, 2, 99, 4, 5, 10]
[8, 8, 99, 8, 8, 10]
```

- **Reversing Rows:** We can reverse each row individually.

```
for i in matrix:  
    i.reverse()
```

The matrix becomes:

```
[99, 10, 0, 0, 99, 1, 0]  
[10, 8, 0, 99, 100, 1]  
[9, 9, 99, 9, 9]  
[10, 0, 0, 99, 7, 6]  
[10, 5, 4, 99, 2, 1]  
[10, 8, 8, 99, 8, 8]
```

This is similar to flipping the matrix horizontally.

- **Reversing the Order of Rows:** We can also reverse the entire matrix.

```
matrix.reverse()
```

The matrix becomes:

```
[10, 8, 8, 99, 8, 8]  
[10, 5, 4, 99, 2, 1]  
[10, 0, 0, 99, 7, 6]  
[9, 9, 99, 9, 9]  
[10, 8, 0, 99, 100, 1]  
[99, 10, 0, 0, 99, 1, 0]
```

This is similar to flipping the matrix vertically.

A multi-dimensional list can always grow, shrink, and change because it is still a Python list.

However, if you append values inconsistently like `matrix[0].append(99)` ; or replace rows with different lengths: `matrix[2] = [1, 2]` , the structure stops being a true matrix and becomes a **jagged list**.

For many mathematical operations such as matrix multiplication, determinants, inverses, and linear algebra, maintaining a proper rectangular matrix is

essential.

2.3. Closest Pair of Points

One interesting application of multi-dimensional lists is storing coordinates. Suppose each point is represented as:

```
points = [[1, 2], [5, 6], [2, 3], [9, 1]]
```

Each point contains an x-coordinate and a y-coordinate.

- First, we compute the distance between two points:

```
def distance(x1, y1, x2, y2):  
    return ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
```

- Next, we search every possible pair of points to find the closest pair.

```
def nearest_points(points):  
    p1, p2 = 0, 1  
  
    shortest_distance = distance(points[p1][0], points[p1]  
[1],  
                                points[p2][0], points[p2]  
[1])  
  
    for i in range(len(points)):  
        for j in range(i + 1, len(points)):  
            d = distance(points[i][0], points[i][1],  
                          points[j][0], points[j][1])  
  
            if d < shortest_distance:  
                shortest_distance = d  
                p1, p2 = i, j  
  
    return p1, p2
```

The algorithm works as follows:

1. Start with two points as the current closest pair.
2. Compute their distance.
3. Compare every possible pair of points.
4. Whenever a shorter distance is found, update the closest pair.
5. Return the indexes of the nearest points.

```
p1, p2 = nearest_points(points)
print(f"Closest points: {points[p1]} and {points[p2]}")
```

Closest points: [1, 2] and [2, 3]

This idea appears in computer graphics, geographic information systems (GIS), robotics, and machine learning.

2.4. Dot Product

One of the most famous operations involving matrices is the **Dot Product**, also known as **Matrix Multiplication**.

Suppose we have the following two matrices:

```
matrix1 = [
    [1, 2, 3, 4, 5],
    [6, 7, 0, 0, 0],
    [0, 1, 0, 0, 0],
    [1, 0, 0, 0, 8],
    [0, 0, 9, 0, 3]
]
```

```
matrix2 = [
    [1, 0, 2, 1, 0],
    [0, 1, 3, 0, 1],
    [1, 1, 0, 2, 0],
    [0, 0, 1, 1, 2],
]
```

```
[2, 0, 0, 1, 1]  
]
```

First and foremost, it is important to distinguish matrix multiplication from **element-wise multiplication**.

- **Element-Wise Multiplication:** Element-wise multiplication multiplies corresponding elements in the same position.

```
# Element-Wise Multiplication  
result = [[matrix1[i][j] * matrix2[i][j] for j in range(len(  
    (matrix1[0])))  
          for i in range(len(matrix1))]  
  
for i in range(len(result)):  
    print(result[i])
```

```
[1, 0, 6, 4, 0]  
[0, 7, 0, 0, 0]  
[0, 1, 0, 0, 0]  
[0, 0, 0, 0, 16]  
[0, 0, 0, 0, 3]
```

This operation is common in image processing and scientific computing.

- Matrix Multiplication (or Dot Product) is very different. Instead of multiplying elements at the same position, we multiply a row from the first matrix with a column from the second matrix.

For matrix multiplication to be possible:

$$(n \times p) \cdot (p \times m)$$

The **inner dimensions** must be the same.

$$(2 \times 3) \cdot (3 \times 4) \quad \checkmark$$
$$2 \times 3 \cdot 2 \times 4 \quad \times$$

The resulting matrix has dimensions: $(n \times m)$


```

rows = len(matrix1)
cols = len(matrix2[0])

result = []

for i in range(rows):
    row = []

    for j in range(cols):
        total = 0

        for k in range(len(matrix2)):
            total += matrix1[i][k] * matrix2[k][j]
        row.append(total)

    result.append(row)

for row in result:
    print(row)

```

```

[14, 5, 12, 16, 15]
[6, 7, 33, 6, 7]
[0, 1, 3, 0, 1]
[17, 0, 2, 9, 8]
[15, 9, 0, 21, 3]

```

The key idea is:

Result = Row i of matrix 1 · Column j of matrix 2

Every value in the result matrix is itself a dot product.

Visualizing One Cell - For example:

$$\text{Result}[0][0] = (1 \times 1) + (2 \times 0) + (3 \times 1) + (4 \times 0) + (5 \times 2) = 14$$

Each cell is produced by multiplying matching elements and summing them together.

2.5. Grading System Using a 3-Dimensional List

Not all multi-dimensional lists are matrices. Sometimes we need three dimensions.

Consider the following grading system:

```
scores = [
    [[11.5, 20.5], [11.0, 22.5], [15, 33.5], [13, 21.5], [15, 2.5]],
    [[4.5, 21.5], [11.0, 22.5], [15, 34.5], [12, 20.5], [14, 11.5]],
    [[6.5, 30.5], [11.4, 11.5], [11, 33.5], [11, 23.5], [10, 2.5]],
    [[6.5, 23.5], [11.4, 32.5], [13, 34.5], [11, 20.5], [16, 11.5]],
    [[8.5, 26.5], [11.4, 52.5], [13, 36.5], [13, 24.5], [16, 2.5]],
    [[11.5, 20.5], [11.4, 42.5], [13, 31.5], [12, 20.5], [16, 6.5]]
]
```

This structure stores:

`scores[i][j][k]`

- $i \rightarrow$ Student;
- $j \rightarrow$ Exam
- $k \rightarrow$ Score Type
- $k = 0 \rightarrow$ Multiple Choice
- $k = 1 \rightarrow$ Essay

Example: `scores[0][1][0]` and `scores[0][1][1]` refers to: **Student 1; Exam 2; Multiple-Choice Score = 11.0; Essay Score = 22.5**

- **Computing Average Scores** - Suppose we want to calculate for every student:
 - Average Multiple-Choice score
 - Average Essay score
 - Overall Average score

```

student_names = ["Student A", "Student B", "Student C",
                 "Student D", "Student E", "Student F"]

for i in range(len(scores)):
    mc_total = 0
    essay_total = 0

    for exam in scores[i]:
        mc_total += exam[0]
        essay_total += exam[1]

    mc_average = mc_total / len(scores[i])
    essay_average = essay_total / len(scores[i])

    overall_average = (mc_total + essay_total) / (len(scores[i]) * 2)

    print(student_names[i])
    print("Multiple Choice:", round(mc_average, 2))
    print("Essay:", round(essay_average, 2))
    print("Overall:", round(overall_average, 2))
    print()

```

Student A
 Multiple Choice: 13.1
 Essay: 20.1
 Overall: 16.6

Student B
 Multiple Choice: 11.3
 Essay: 22.1
 Overall: 16.7

Student C
 Multiple Choice: 9.98
 Essay: 20.3
 Overall: 15.14

Student D
 Multiple Choice: 11.58

Essay: 24.5
Overall: 18.04

Student E
Multiple Choice: 12.38
Essay: 28.5
Overall: 20.44

Student F
Multiple Choice: 12.78
Essay: 24.3
Overall: 18.54

This type of structure is commonly used in: Student grading systems; Scientific experiments; Financial records; Sports statistics; Machine Learning datasets, where data naturally has more than two dimensions.

Summary

Multi-Dimensional Lists extend ordinary lists by allowing lists to contain other lists. A 1-dimensional list forms a **Vector**, a 2-dimensional list forms a **Matrix**, and higher-dimensional structures are often called **Tensors**.

With Multi-Dimensional Lists, we can represent tables, spreadsheets, images, calendars, game boards, distance maps, student records, and machine learning datasets.

Common operations include:

- Traversing rows and columns
- Computing row and column totals
- Finding maximum or minimum values
- Accessing diagonal elements
- Sorting rows or columns
- Matrix multiplication (Dot Product)
- Working with higher-dimensional data structures

Although they are conceptually simple—just "lists inside lists"—multi-dimensional lists are one of the most powerful and widely used data structures in Python, forming the foundation for many advanced topics in mathematics,

data science, artificial intelligence, computer graphics, and scientific computing.

References

<https://www.geeksforgeeks.org/python/multi-dimensional-lists-in-python/>

<https://www.youtube.com/watch?v=UyA6g4hKpTU>

Hand-On Exercises:

</> [Ex 2. Python Exercises](#)

Next Section:

</> [26. Python: Read and Calculate IP Address](#)